

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM SISTEMAS DE INFORMAÇÃO

MATHEUS BAHRY DE LIMA

**PRICEFINDER – COMPARADOR DE VALORES
DE PRODUTOS**

Mogi das Cruzes

2026

MATHEUS BAHRY DE LIMA

**PRICEFINDER – COMPARADOR DE VALORES
DE PRODUTOS**

Projeto para finalização do curso de Bacharelado em Sistemas de Informação da Universidade de Mogi das Cruzes, sob orientação do Prof. Alessandro Aparecido da Silva e coorientação do Prof. Oscar Alves Evangelista, contendo Introdução detalhando as tecnologias utilizadas, com as respectivas citações e referências de pesquisa.

Mogi das Cruzes

2026

INTRODUÇÃO

O crescimento do comércio eletrônico tem transformado significativamente a forma como consumidores pesquisam e realizam compras pela internet. Com a grande quantidade de plataformas de vendas disponíveis, como marketplaces e lojas virtuais, tornou-se comum que usuários precisem acessar diferentes sites para comparar preços e identificar promoções. Nesse contexto, surgem ferramentas que buscam centralizar essas informações, facilitando o processo de pesquisa e tomada de decisão por parte do consumidor. Diante desse cenário, este projeto propõe o desenvolvimento de uma aplicação web que funcione como um catálogo de produtos provenientes de diferentes plataformas de comércio eletrônico, permitindo ao usuário visualizar ofertas e identificar as melhores oportunidades de compra de maneira rápida e prática.

Para o desenvolvimento da aplicação será utilizada a linguagem de programação Java, amplamente empregada no desenvolvimento de sistemas corporativos devido à sua portabilidade, segurança e grande ecossistema de bibliotecas e frameworks (DEITEL; DEITEL, 2017). O backend da aplicação será construído utilizando o framework Spring Boot, que simplifica a criação de aplicações baseadas em Spring e permite o desenvolvimento de APIs de forma rápida e eficiente (WALLS, 2022).

Além disso, será utilizado o Spring Security, responsável pela implementação de mecanismos de autenticação e autorização na aplicação, garantindo maior segurança no acesso às funcionalidades do sistema. A autenticação será realizada por meio de JSON Web Token (JWT), um padrão aberto amplamente utilizado para transmissão segura de informações entre partes em formato de token (JONES; BRADLEY; SAKIMURA, 2015).

No desenvolvimento da interface do usuário será utilizada a biblioteca React, que permite a construção de interfaces modernas baseadas em componentes reutilizáveis e facilita o desenvolvimento de aplicações web dinâmicas (BANKS; PORCELLO, 2020). Para gerenciamento de estado da aplicação será utilizada a biblioteca Zustand, que possibilita o controle eficiente de dados compartilhados entre diferentes componentes da interface.

Para a estilização da interface serão utilizados Tailwind CSS e DaisyUI, ferramentas que permitem a criação de layouts responsivos e modernos por meio de componentes reutilizáveis e classes utilitárias. O sistema utilizará ainda o banco de dados PostgreSQL, um sistema gerenciador de banco de dados relacional amplamente utilizado em aplicações web devido à sua confiabilidade, desempenho e suporte a recursos avançados de manipulação de dados (STONEBRAKER; ROWE, 1986). Dessa forma, a combinação dessas tecnologias permitirá o desenvolvimento de uma aplicação web moderna, segura e escalável, capaz de centralizar informações de produtos e preços provenientes de diferentes plataformas de comércio eletrônico.

Para o desenvolvimento do backend da aplicação foi adotado o módulo Spring WebFlux, disponível no ecossistema Spring Framework, que implementa o paradigma de programação reativa baseado na especificação Reactive Streams.

Diferentemente do modelo tradicional de servidor bloqueante, em que cada requisição ocupa uma thread durante todo o seu ciclo de vida, o Spring WebFlux utiliza um modelo assíncrono e não bloqueante, permitindo que poucas threads atendam um grande volume de requisições simultâneas de forma eficiente (WALLS, 2022). Esse modelo é especialmente adequado para aplicações que realizam chamadas intensivas a serviços externos, como APIs de terceiros, pois evita que o servidor fique ocioso aguardando respostas externas. Os tipos fundamentais do Spring WebFlux são *Mono* e *Flux*, provenientes da biblioteca Project Reactor. O *Mono* representa zero ou um elemento retornado de forma assíncrona, enquanto o *Flux* representa uma sequência de zero a N elementos. No projeto PriceFinder, todos os retornos de endpoints, consultas ao banco de dados e chamadas às APIs externas utilizam esses tipos reativos, garantindo consistência no fluxo de dados ao longo de toda a aplicação.

Para a comunicação com o banco de dados PostgreSQL de forma reativa, foi utilizada a especificação R2DBC (Reactive Relational Database Connectivity), que define uma API não bloqueante para acesso a bancos de dados relacionais. O Spring Data R2DBC fornece a implementação dessa especificação integrada ao ecossistema Spring, permitindo que as operações de persistência retornem tipos reativos em vez de bloquear a thread até a conclusão da consulta. Os repositórios da aplicação estendem a interface *ReactiveCrudRepository*, que provê operações básicas de criação, leitura, atualização e exclusão de forma reativa, garantindo que toda a cadeia de processamento da aplicação permaneça não bloqueante.

A segurança das senhas dos usuários é garantida pelo algoritmo BCrypt, implementado pela classe *BCryptPasswordEncoder* do Spring Security. O BCrypt é uma função de hash adaptativa baseada na cifra Blowfish, projetada especificamente para armazenamento seguro de senhas. Sua principal característica é o fator de custo configurável, que permite ajustar o número de iterações do algoritmo à medida que o poder computacional aumenta, tornando ataques de força bruta progressivamente mais custosos (OWASP, 2021). No projeto PriceFinder, a senha é transformada em hash BCrypt no momento do cadastro, por meio do método *setPassword()* da entidade *User*, e o valor original jamais é armazenado no banco de dados. Na autenticação, a comparação é feita entre a senha informada e o hash armazenado utilizando o método *matches()* do encoder.

O frontend do projeto foi criado utilizando Vite, uma ferramenta moderna de build para aplicações web que oferece um servidor de desenvolvimento com recarga instantânea de módulos (HMR — Hot Module Replacement) e tempo de inicialização significativamente menor em comparação com ferramentas tradicionais (EVAN YOU, 2021). O gerenciamento de rotas da aplicação é realizado pela biblioteca React Router, que possibilita a navegação entre diferentes páginas de uma Single Page Application (SPA) sem recarregar o navegador, sendo utilizada para definir as rotas, implementar redirecionamentos condicionais baseados no estado de autenticação do usuário e gerenciar a navegação entre as telas da aplicação.

DESENVOLVIMENTO DO SISTEMA

Estrutura Analítica do Projeto (EAP / WBS)

A Estrutura Analítica do Projeto (EAP), também conhecida como Work Breakdown Structure (WBS), é uma técnica de gerenciamento de projetos utilizada para organizar e dividir o desenvolvimento de um sistema em partes menores e mais gerenciáveis. Por meio da EAP é possível estruturar as etapas do projeto, facilitando o planejamento, execução e acompanhamento das atividades ao longo do desenvolvimento.

No projeto **PriceFinder**, a EAP foi elaborada com o objetivo de organizar as etapas relacionadas ao desenvolvimento da aplicação web, considerando desde o planejamento do projeto até a implantação do sistema em ambiente de produção.

A estrutura do projeto foi dividida nos seguintes módulos principais:

- Gerenciamento do Projeto
- Backend
- Integração com APIs
- Banco de Dados
- Frontend
- Implantação

Cada um desses módulos possui pacotes de trabalho responsáveis por etapas específicas do desenvolvimento do sistema.

A estrutura analítica do projeto pode ser visualizada na Figura 1.

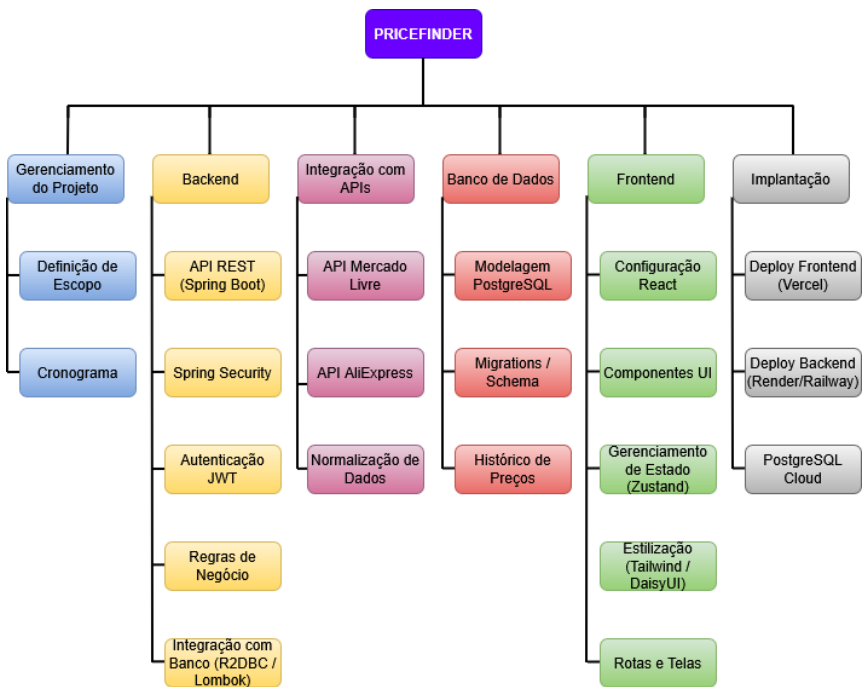


Figura 1 – Estrutura analítica do projeto PriceFinder

Fonte: Elaborado pelo autor (2026)

1. Gerenciamento do Projeto

Este módulo contempla as atividades relacionadas ao planejamento e organização do desenvolvimento do sistema.

1.1 Definição de Escopo

Nesta etapa foram definidos os objetivos do projeto, os requisitos funcionais e não funcionais do sistema, bem como as restrições e limites da aplicação. Também foram identificadas as principais funcionalidades do sistema, como busca e comparação de preços, histórico de preços e salvamento de promoções.

1.2 Cronograma do Projeto

Refere-se ao planejamento das etapas de desenvolvimento, incluindo a organização das atividades ao longo do semestre, definição de marcos de entrega e acompanhamento da evolução do projeto.

2. Backend

O backend do sistema é responsável por processar as requisições enviadas pelo frontend, aplicar as regras de negócio e realizar a comunicação com o banco de dados e APIs externas.

2.1 API REST com Spring Boot

Foi utilizada a framework Spring Boot para o desenvolvimento da API RESTful do sistema. Essa API será responsável por disponibilizar endpoints que permitirão a busca de produtos, comparação de preços e gerenciamento das promoções salvas pelos usuários.

2.2 Segurança com Spring Security

Para garantir a proteção das informações e o controle de acesso às funcionalidades do sistema, foi implementado o Spring Security, responsável pela autenticação e autorização dos usuários.

2.3 Autenticação com JWT

O sistema utilizará autenticação baseada em JSON Web Token (JWT), permitindo que o usuário realize login e receba um token que será utilizado para validar suas requisições na aplicação.

2.4 Regras de Negócio

Nesta etapa serão implementadas as principais funcionalidades do sistema, incluindo:

- Busca de produtos em diferentes marketplaces
- Comparação de preços
- Salvamento de promoções
- Consulta ao histórico de preços

2.5 Comunicação com Banco de Dados

A comunicação com o banco de dados PostgreSQL será realizada utilizando R2DBC, permitindo operações reativas e assíncronas. Além disso, será utilizada a biblioteca Lombok para reduzir código repetitivo no desenvolvimento da aplicação.

3. Integração com APIs de Marketplaces

Esta etapa é responsável por permitir que o sistema obtenha informações de produtos a partir de plataformas de comércio eletrônico.

3.1 Integração com API do Mercado Livre

Será realizada a integração com a API pública do Mercado Livre para coletar informações como nome do produto, preço, imagens e link de compra.

3.2 Integração com API do AliExpress

Também será realizada integração com serviços que disponibilizem dados de produtos do AliExpress ou plataformas semelhantes.

3.3 Normalização de Dados

Como cada API possui um formato de resposta diferente, será necessário implementar um processo de normalização de dados, permitindo que as informações sejam apresentadas de forma padronizada dentro do sistema.

4. Banco de Dados

O banco de dados é responsável pelo armazenamento das informações persistentes da aplicação.

4.1 Modelagem do Banco de Dados

Foi projetado um modelo relacional utilizando PostgreSQL para armazenar dados de usuários, produtos, histórico de preços e promoções salvas.

4.2 Migrations e Versionamento do Schema

Serão utilizadas migrations para manter o controle de versões da estrutura do banco de dados, garantindo consistência entre os ambientes de desenvolvimento e produção.

4.3 Histórico de Preços

Será implementada uma estrutura específica para armazenar as variações de preço de cada produto ao longo do tempo, permitindo a análise de tendências e mudanças de valor.

5. Frontend

O frontend do sistema será responsável pela interface com o usuário e pela comunicação com a API.

5.1 Configuração do Projeto React

Será criado o projeto frontend utilizando React.js, com organização de pastas, dependências e configuração do ambiente de desenvolvimento.

5.2 Desenvolvimento de Componentes de Interface

Serão desenvolvidos componentes reutilizáveis, como:

- Cards de produtos
- Listas de resultados
- Formulários
- Modais

5.3 Gerenciamento de Estado com Zustand

O gerenciamento global de estado será realizado com a biblioteca Zustand, permitindo compartilhar informações entre diferentes componentes da aplicação.

5.4 Estilização com Tailwind CSS e DaisyUI

A interface será estilizada utilizando Tailwind CSS e DaisyUI, possibilitando a criação de layouts modernos, responsivos e de fácil manutenção.

5.5 Rotas e Telas da Aplicação

Serão implementadas as principais telas do sistema:

- Tela de busca de produtos
- Tela de comparação de preços
- Tela de histórico de preços
- Tela de promoções salvas

DIAGRAMA DE CLASSES

A Figura 2 apresenta o diagrama de classes do sistema PriceFinder, representando as principais entidades e serviços da aplicação, seus atributos, métodos e relacionamentos.



Figura 2 – Diagrama de Classes do projeto PriceFinder

Fonte: Elaborado pelo autor (2026)

BPMN

A Figura 3 apresenta o diagrama de atividades do fluxo principal do sistema, descrevendo o processo de busca e comparação de preços. O fluxo contempla três participantes: o usuário, o sistema backend e a API externa do eBay. O processo se inicia com o usuário informando o termo de busca, seguido da verificação de autenticação via token JWT. Após validado, o backend constrói e envia a requisição à API do eBay, que retorna os produtos encontrados. Os dados são então normalizados e ordenados pelo sistema antes de serem exibidos ao usuário.

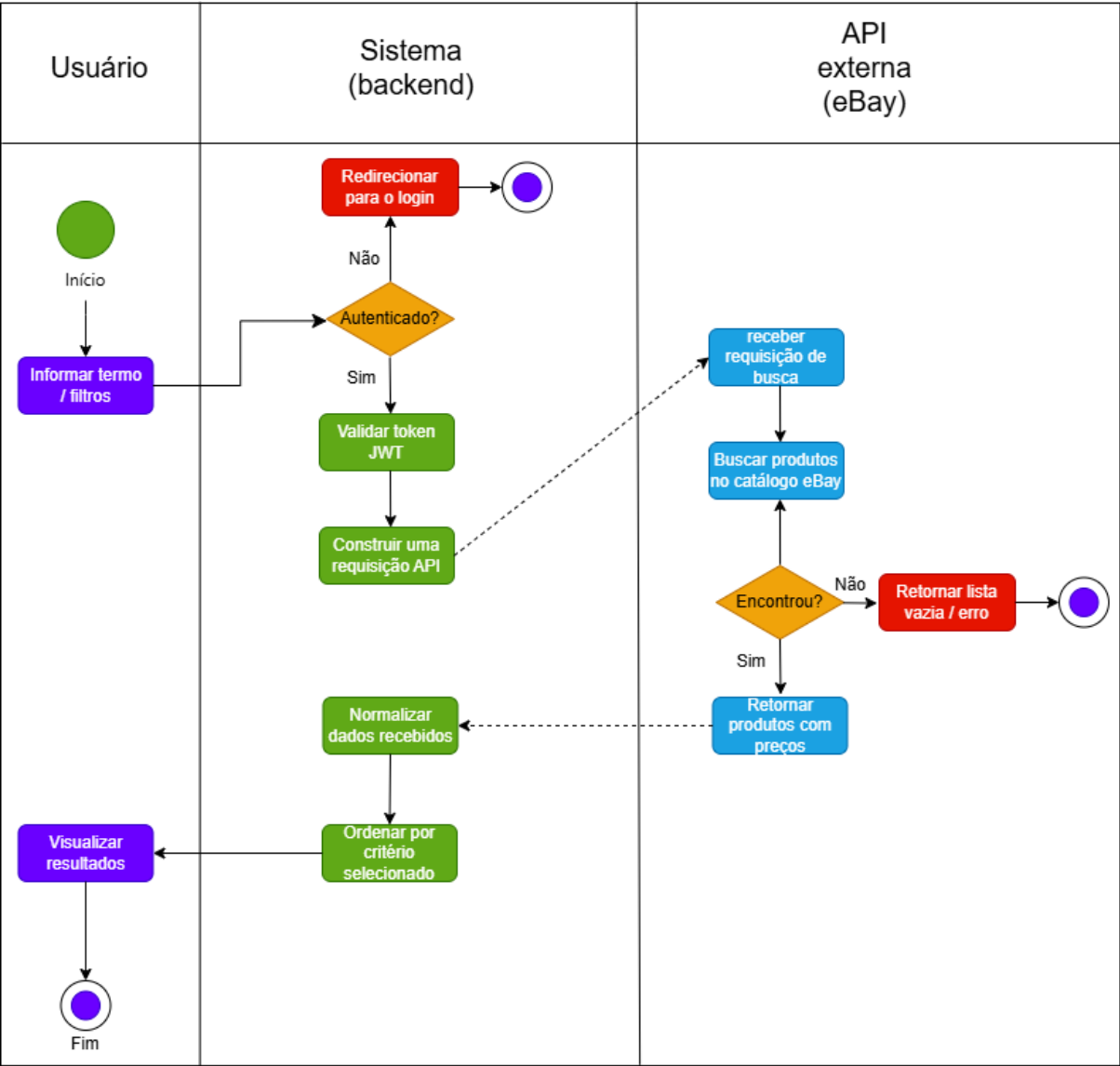


Figura 3 – Diagrama de Atividades – Busca e comparação de preços

Fonte: Elaborado pelo autor (2026)

FUNCIONALIDADES PRINCIPAIS DO SISTEMA

O sistema PriceFinder foi projetado com três funcionalidades principais que representam o núcleo da aplicação.

Busca e Comparação de Preços

Esta funcionalidade permitirá que o usuário pesquise um produto e visualize resultados provenientes de diferentes marketplaces. O sistema realizará consultas nas APIs integradas, coletando dados como nome do produto, preço, imagem e link de compra.

Após a coleta dessas informações, o sistema organizará os resultados de forma comparativa, permitindo que o usuário identifique rapidamente qual plataforma oferece o melhor preço disponível.

Essa funcionalidade será implementada principalmente no backend, responsável pela comunicação com as APIs externas, e no frontend, responsável pela exibição dos resultados ao usuário.

Histórico de Preços

O histórico de preços permitirá que o usuário acompanhe a variação de preço de um produto ao longo do tempo. Para isso, o sistema armazenará registros de preços coletados durante as consultas realizadas.

Com essas informações, será possível apresentar ao usuário um histórico que demonstre se o produto está mais caro ou mais barato em relação a períodos anteriores.

Essa funcionalidade auxilia o usuário a tomar decisões mais informadas antes de realizar uma compra.

Sistema de Salvamento de Promoções

O sistema também permitirá que o usuário salve promoções ou produtos de interesse dentro da plataforma. Para utilizar essa funcionalidade, será necessário que o usuário esteja autenticado no sistema.

As promoções salvas ficarão associadas ao perfil do usuário no banco de dados, permitindo acesso posterior às ofertas selecionadas.

Essa funcionalidade aumenta a usabilidade do sistema e incentiva o retorno do usuário à plataforma.

ARQUITETURA DO SISTEMA

A arquitetura do sistema PriceFinder foi projetada seguindo o modelo de separação entre frontend e backend, também conhecido como arquitetura cliente-servidor.

No frontend, será utilizada a biblioteca React, responsável pela construção da interface do usuário e pela comunicação com a API do sistema. O frontend realizará requisições HTTP para o backend sempre que o usuário realizar buscas, acessar o histórico de preços ou salvar promoções.

O backend será desenvolvido utilizando Spring Boot, sendo responsável por processar as requisições recebidas, aplicar as regras de negócio e realizar a comunicação com o banco de dados PostgreSQL.

Além disso, o backend também será responsável pela integração com APIs externas de marketplaces, coletando informações sobre produtos disponíveis nas plataformas de comércio eletrônico.

O banco de dados PostgreSQL armazenará informações relacionadas aos usuários, histórico de preços e promoções salvas.

Dessa forma, a arquitetura do sistema permite uma separação clara de responsabilidades entre as camadas da aplicação, facilitando a manutenção, escalabilidade e evolução do sistema.

A arquitetura do sistema pode ser visualizada na Figura 4.

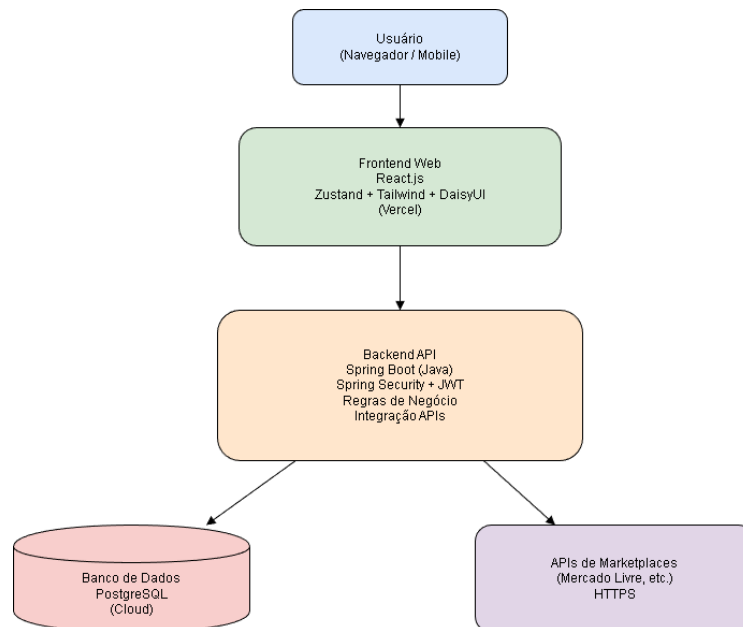


Figura 4 – Arquitetura do projeto PriceFinder

Fonte: Elaborado pelo autor (2026)

ARQUITETURA DE IMPLANTAÇÃO

A implantação do sistema será realizada em ambiente de nuvem, utilizando serviços modernos de hospedagem de aplicações web.

O frontend desenvolvido em React será hospedado na plataforma Vercel, que permite integração contínua com repositórios GitHub e deploy automático a cada atualização do projeto.

O backend desenvolvido em Spring Boot será hospedado em plataformas de cloud computing, como Render ou Railway, que permitem a execução de aplicações Java em ambiente escalável.

O banco de dados PostgreSQL será hospedado em ambiente de nuvem, garantindo disponibilidade e segurança no armazenamento das informações.

A comunicação entre os serviços será realizada por meio de conexões seguras utilizando o protocolo HTTPS.

ELEMENTO TEXTUAL

1. Requisitos do Sistema e Protótipos de Navegação

Esta seção apresenta os requisitos funcionais e não funcionais do sistema PriceFinder, bem como os protótipos de navegação da aplicação, representados por meio de capturas de tela das principais telas desenvolvidas.

1.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer ao usuário. Para o projeto PriceFinder, foram identificados os seguintes requisitos funcionais:

RF01 — O sistema deve permitir que o usuário realize seu cadastro informando e-mail e senha.

RF02 — O sistema deve permitir que o usuário realize login com e-mail e senha cadastrados.

RF03 — O sistema deve autenticar o usuário por meio de token JWT, retornando-o ao cliente após login ou cadastro bem-sucedido.

RF04 — O sistema deve armazenar a senha do usuário de forma criptografada no banco de dados, utilizando o algoritmo BCrypt.

RF05 — O sistema deve controlar o acesso às funcionalidades por meio de roles de autorização (OPERADOR e ADMIN).

RF06 — O sistema deve permitir que o usuário autenticado realize buscas de produtos por palavra-chave.

RF07 — O sistema deve permitir que o usuário autenticado busque produtos dentro de uma faixa de preço especificada.

RF08 — O sistema deve permitir que o usuário autenticado ordene os resultados de busca por critérios como melhor correspondência e menor preço.

RF09 — O sistema deve exibir os resultados de busca em formato de cards contendo imagem, título, preço, condição e informação sobre frete grátis.

RF10 — O sistema deve permitir que o usuário visualize os detalhes de um produto selecionado em uma janela modal.

RF11 — O sistema deve redirecionar o usuário para a página de login caso tente acessar rotas protegidas sem estar autenticado.

RF12 — O sistema deve permitir que o usuário realize logout, encerrando sua sessão na aplicação.

1.2 Requisitos Não Funcionais

Os requisitos não funcionais definem as características de qualidade do sistema, como desempenho, segurança e usabilidade.

RNF01 - O backend deve ser desenvolvido utilizando programação reativa (Spring WebFlux e R2DBC), garantindo que nenhuma operação de I/O bloqueie threads do servidor.

RNF02 - A senha do usuário deve ser armazenada exclusivamente como hash BCrypt, nunca em texto simples.

RNF03 - O token JWT deve ter validade máxima de dez horas, após o qual o usuário deverá realizar novo login.

RNF04 - A comunicação entre frontend e backend deve ser realizada por meio de HTTPS em ambiente de produção.

RNF05 - O sistema deve ser responsivo, adaptando-se a diferentes tamanhos de tela (desktop e dispositivos móveis).

RNF06 - Os tokens de acesso às APIs externas (eBay e Mercado Livre) devem ser renovados automaticamente pelo sistema, sem intervenção do usuário.

RNF07 - O sistema deve retornar respostas de erro adequadas (HTTP 401, 409, 500) com mensagens descritivas em caso de falha de autenticação, conflito de cadastro ou erro interno.

RNF08 - O banco de dados deve utilizar identificadores UUID gerados nativamente pelo PostgreSQL por meio da extensão pgcrypto.

1.3 Protótipos de Navegação

As figuras a seguir apresentam as principais telas da aplicação PriceFinder, demonstrando o fluxo de navegação do usuário desde o acesso inicial até a utilização das funcionalidades de busca.

The screenshot shows the 'Criar Conta' (Create Account) form in the PriceFinder application. The form is centered on a dark background. It includes two input fields for 'Email' and 'Senha' (Password), followed by a red 'Cadastrar' (Register) button. The top navigation bar contains the 'PriceFinder' logo, a 'Home' link, a 'Buscar por Preço' link, and a red 'Logout' button.

Figura 5 – Tela de Cadastro

Fonte: Elaborado pelo autor (2026)

The screenshot shows the 'Login' form in the PriceFinder application. The form is centered on a dark background. It includes two input fields for 'Email' and 'Senha' (Password), followed by a blue 'Entrar' (Enter) button. The top navigation bar contains the 'PriceFinder' logo, a 'Home' link, a 'Buscar por Preço' link, and a red 'Logout' button.

Figura 6 – Tela de Login

Fonte: Elaborado pelo autor (2026)

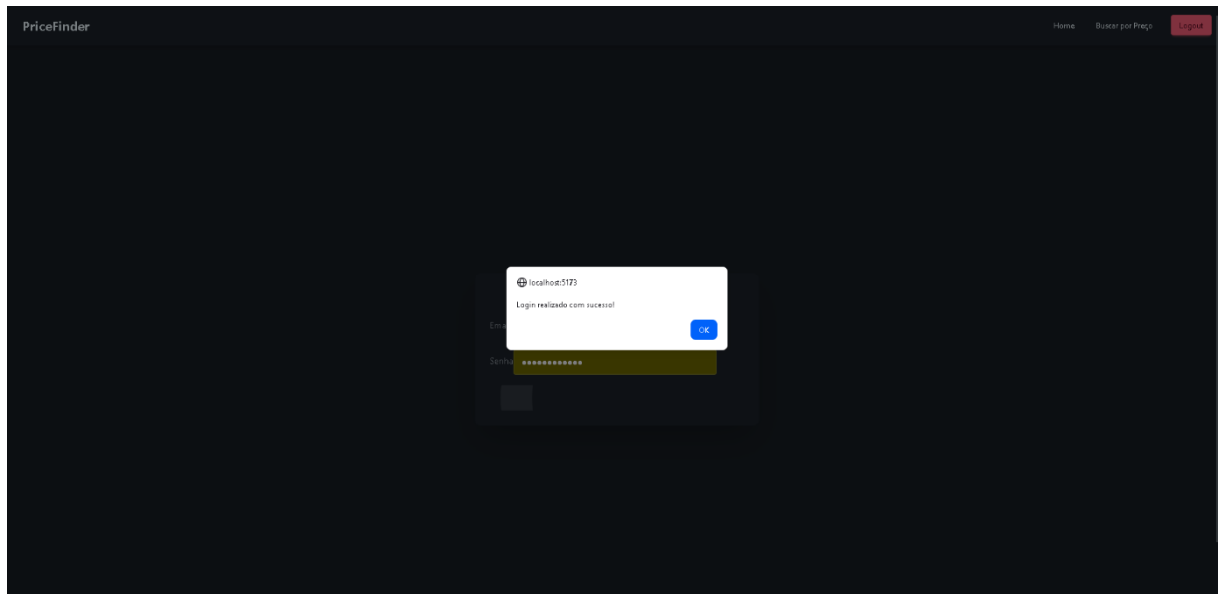


Figura 7 – Tela de Login/Sucesso

Fonte: Elaborado pelo autor (2026)

Validação de Formulários

A aplicação PriceFinder implementa validação de campos nos formulários de cadastro e login, garantindo que os dados inseridos pelo usuário estejam corretos antes de serem enviados ao servidor.

No formulário de cadastro, são aplicadas as seguintes validações:

- Primeiro nome e sobrenome são campos obrigatórios, não sendo permitido o envio do formulário com esses campos em branco;
- O campo de e-mail verifica se o valor informado corresponde a um formato válido de endereço eletrônico;
- A senha deve conter no mínimo 8 caracteres, ao menos uma letra maiúscula e ao menos um número;
- O campo de confirmação de senha verifica se o valor coincide com a senha informada.

No formulário de login, são aplicadas as seguintes validações:

- O campo de e-mail é obrigatório e deve estar em formato válido;
- O campo de senha é obrigatório e deve conter no mínimo 8 caracteres.

Em ambos os formulários, as mensagens de erro são exibidas imediatamente abaixo do campo correspondente, com destaque visual em vermelho, sem necessidade de recarregar a página. O envio da requisição ao backend só ocorre após todos os campos passarem pela validação no frontend.

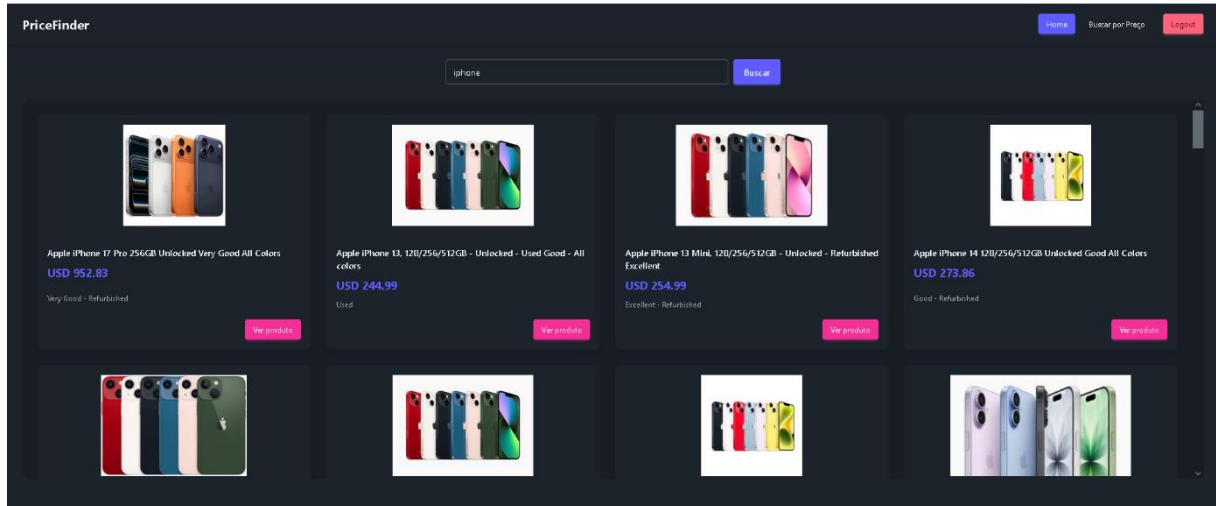


Figura 8 – Tela Principal Busca por Palavra-chave

Fonte: Elaborado pelo autor (2026)

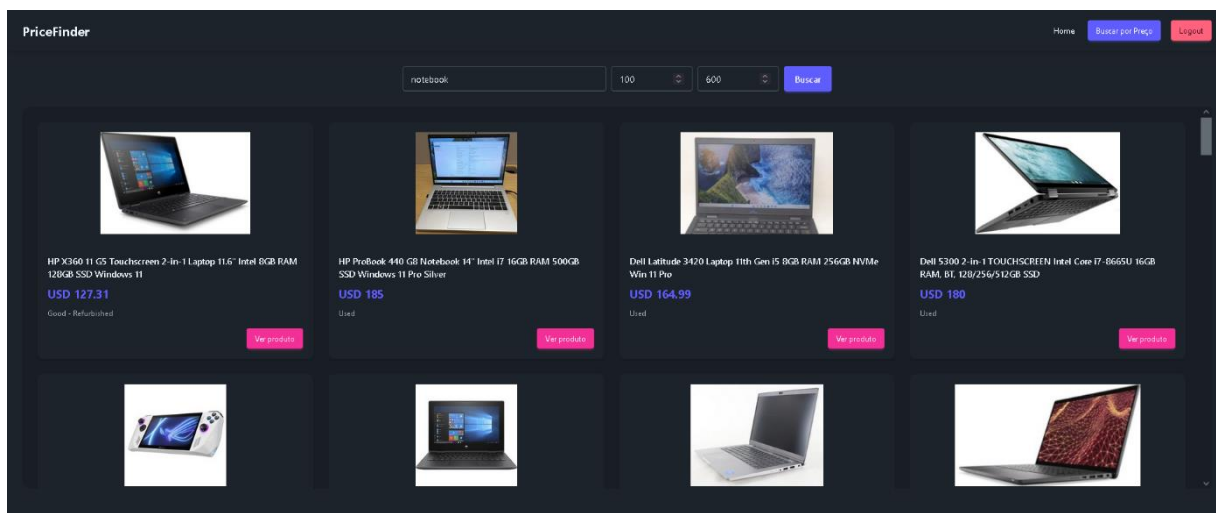


Figura 9 – Tela de Busca por Faixa de Preço

Fonte: Elaborado pelo autor (2026)

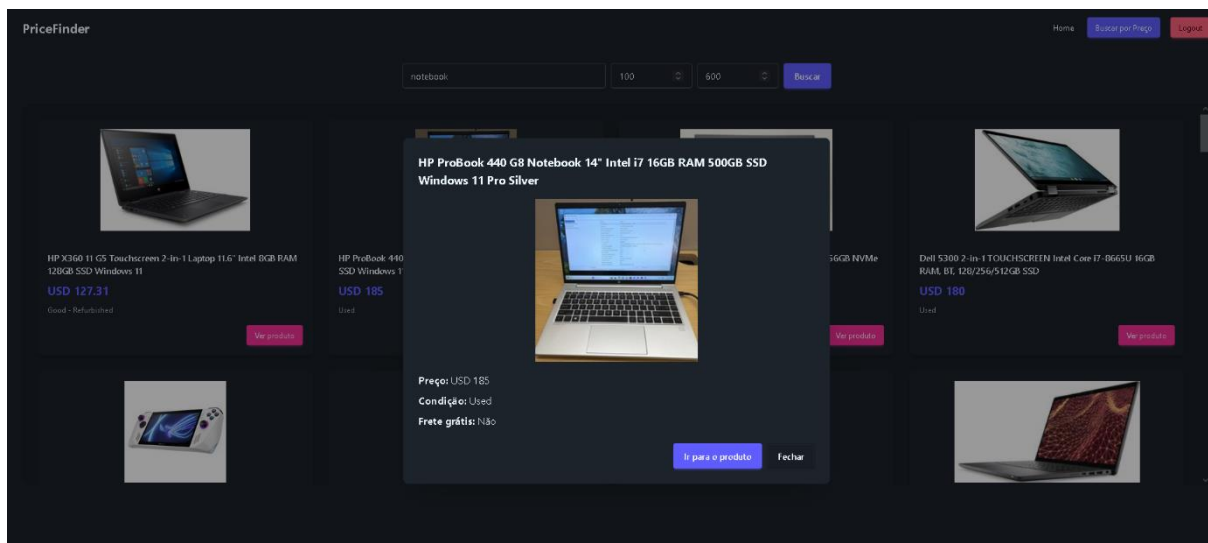


Figura 10 – Modal de Detalhes do Produto

Fonte: Elaborado pelo autor (2026)

2. Funcionalidades Desenvolvidas na Terceira Entrega

A terceira entrega do projeto PriceFinder contemplou o desenvolvimento de duas frentes principais: o módulo de autenticação e autorização de usuários, implementado como funcionalidade adicional da entrega, e a integração com a API do eBay, responsável pela busca e exibição de produtos na aplicação. Esta seção descreve em detalhes as funcionalidades implementadas, abrangendo tanto o backend quanto o frontend do sistema.

2.1 Cadastro de Usuários

O cadastro de novos usuários é realizado por meio do endpoint POST /auth/signup. Ao receber uma requisição de cadastro contendo e-mail e senha, o sistema verifica previamente se já existe um usuário cadastrado com o mesmo e-mail. Caso o e-mail já esteja em uso, o sistema retorna o status HTTP 409 (Conflict), impedindo duplicidades no banco de dados.

Quando o e-mail ainda não está cadastrado, o sistema instancia um novo objeto User e invoca o método setPassword(), que aplica o algoritmo BCrypt à senha fornecida antes de armazená-la no banco de dados. Em nenhum momento a senha em texto simples é persistida. Após salvar o usuário, o sistema associa automaticamente a role OPERADOR ao novo usuário por meio do serviço UserRoleService, que verifica a inexistência de duplicatas antes de criar o vínculo. Por fim, um token JWT é gerado e retornado ao cliente junto com as informações do usuário criado, permitindo que o usuário já inicie sua sessão imediatamente após o cadastro.

2.2 Autenticação por Login

O login de usuários é realizado por meio do endpoint POST /auth/login. O sistema busca o usuário pelo e-mail informado e, caso não seja encontrado, retorna um erro de credenciais inválidas sem revelar ao cliente se o e-mail existe ou não no sistema, prática recomendada para evitar enumeração de usuários.

Quando o usuário é encontrado, a senha fornecida é comparada com o hash armazenado utilizando o método `matches()` do `BCryptPasswordEncoder`. Em caso de correspondência, o sistema recupera todas as roles associadas ao usuário, gera um token JWT contendo o e-mail, o identificador do usuário e as roles como claims customizadas, e retorna esse token ao cliente com validade de dez horas.

2.3 Geração e Validação de Tokens JWT

A geração e validação de tokens JWT é centralizada na classe `JWTUtil`. O token é assinado utilizando o algoritmo HMAC-SHA256 (HS256) com uma chave secreta configurada no arquivo de propriedades da aplicação. O payload do token contém o e-mail do usuário como subject, o identificador único (UUID), as roles de autorização e os campos padrão de emissão e expiração.

A validação do token ocorre a cada requisição autenticada por meio do componente `JWTAuthenticationManager`, que implementa a interface `ReactiveAuthenticationManager` do Spring Security WebFlux. O processo consiste em extrair o token do cabeçalho Authorization (prefixo Bearer), validar a assinatura e a expiração, recuperar o usuário no banco de dados e construir o objeto de autenticação com as autoridades correspondentes.

2.4 Controle de Acesso por Roles

O sistema implementa controle de acesso baseado em roles por meio do Spring Security WebFlux. A configuração de segurança, definida na classe `SecurityConfig`, estabelece que os endpoints de autenticação (/auth/**) são públicos e acessíveis sem token, enquanto os endpoints de produtos (/api/ebay/products/**) exigem que o usuário possua a role OPERADOR ou ADMIN. O sistema conta com dois perfis de acesso: OPERADOR, atribuído automaticamente a todo novo usuário cadastrado, e ADMIN, reservado a administradores e criado por meio do endpoint POST /auth/signup-admin.

2.5 Persistência de Dados de Autenticação

As informações de autenticação são persistidas em três tabelas no banco de dados PostgreSQL. A tabela `users` armazena o identificador UUID gerado nativamente pelo banco por meio da extensão `pgcrypto`, o e-mail do usuário, o hash BCrypt da senha, o status da conta (ACTIVE ou INACTIVE) e os timestamps de criação e último login. A tabela `roles` armazena os perfis de acesso disponíveis no sistema. A tabela `user_roles` é uma tabela associativa que vincula usuários às suas roles, com restrições de chave estrangeira e exclusão em cascata, garantindo a integridade referencial dos dados.

2.6 Interface de Autenticação no Frontend

No frontend, as telas de login e cadastro foram desenvolvidas em React utilizando componentes do DaisyUI para estilização. O gerenciamento do estado de autenticação é realizado pelo store `useAuthStore`, implementado com a biblioteca Zustand. Após um login ou cadastro bem-sucedido, o token JWT, as informações do usuário e a role são armazenados no `localStorage` do navegador, permitindo que a sessão seja restaurada automaticamente ao recarregar a página.

O controle de acesso às rotas da aplicação é realizado no componente `App.jsx`, que utiliza o React Router para redirecionar automaticamente para a tela de login quando o usuário tenta acessar uma rota protegida sem estar autenticado. A instância do Axios, configurada em `axiosInstance.config.js`, injeta automaticamente o token JWT no cabeçalho `Authorization` de todas as requisições realizadas ao backend, dispensando o tratamento manual do token em cada chamada.

2.7 Integração com a API do eBay

A integração com a API do eBay foi realizada por meio da eBay Browse API, que disponibiliza endpoints para busca e consulta de produtos na plataforma. A autenticação com a API do eBay utiliza o fluxo OAuth 2.0 Client Credentials, em que o sistema obtém automaticamente um token de acesso no momento da inicialização da aplicação, por meio do método anotado com `@PostConstruct` na classe `EbayAuthService`. O token é armazenado em memória e renovado automaticamente por meio de tarefas agendadas com a anotação `@Scheduled`, executadas a cada cinco minutos e a cada hora, garantindo que o token esteja sempre válido sem necessidade de intervenção manual.

O serviço `EbayProductService` é responsável por realizar as chamadas à API do eBay utilizando o `WebClient` de forma reativa. Foram implementadas quatro modalidades de busca: por palavra-chave, por faixa de preço com filtro `price:[min..max]`, por ordenação (melhor correspondência, menor preço, mais recente) e por identificador de produto. Os resultados retornados pela API são mapeados para a entidade `Product`, que padroniza as informações independentemente da origem, contendo campos como título, preço, moeda, link de compra, imagem, condição e indicador de frete grátis.

2.8 Exibição de Produtos no Frontend

No frontend, os resultados de busca são gerenciados pelo store `useProductStore`, implementado com Zustand, que centraliza as ações de busca e o estado de carregamento e erro. A tela principal (Home) permite a busca por palavra-chave, enquanto a tela `SearchByPrice` permite a busca combinando palavra-chave, preço mínimo e preço máximo. Os resultados são exibidos em um grid responsivo de cards contendo imagem, título, preço, moeda e condição do produto. Ao clicar em um card, uma janela modal exibe informações detalhadas do produto, incluindo preço original quando disponível, indicador de frete grátis e um botão de redirecionamento direto para a página do produto na plataforma do eBay.

FUNCIONALIDADES IMPLEMENTADAS NO SISTEMA

A presente seção descreve as funcionalidades implementadas no sistema PriceFinder, abrangendo tanto o frontend quanto o backend da aplicação.

Verificação de Identidade por OTP

O sistema implementa um mecanismo de verificação de identidade por meio de código OTP (One-Time Password) enviado ao e-mail do usuário após o cadastro ou login. Ao concluir o preenchimento do formulário, o usuário é redirecionado para uma tela de verificação onde deve informar o código de seis dígitos recebido em seu e-mail, com validade de cinco minutos. Essa funcionalidade adiciona uma camada extra de segurança ao processo de autenticação, garantindo que apenas o proprietário do e-mail cadastrado possa acessar a plataforma.

Figura 11 – Tela de Cadastro

Fonte: Elaborado pelo autor (2026)

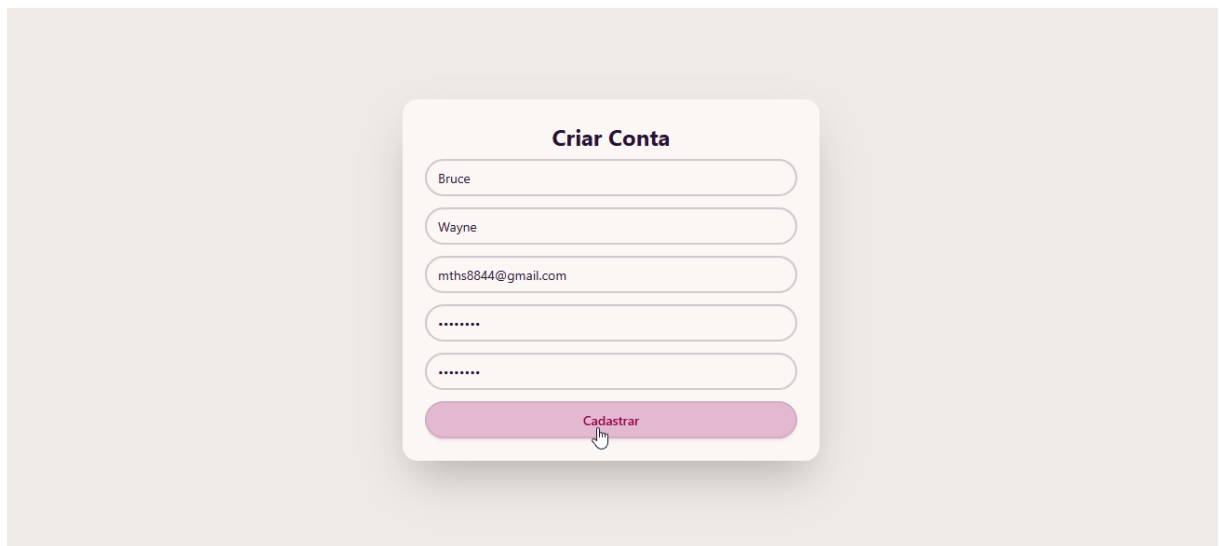


Figura 12 – Recebimento de Token no email

Fonte: Elaborado pelo autor (2026)

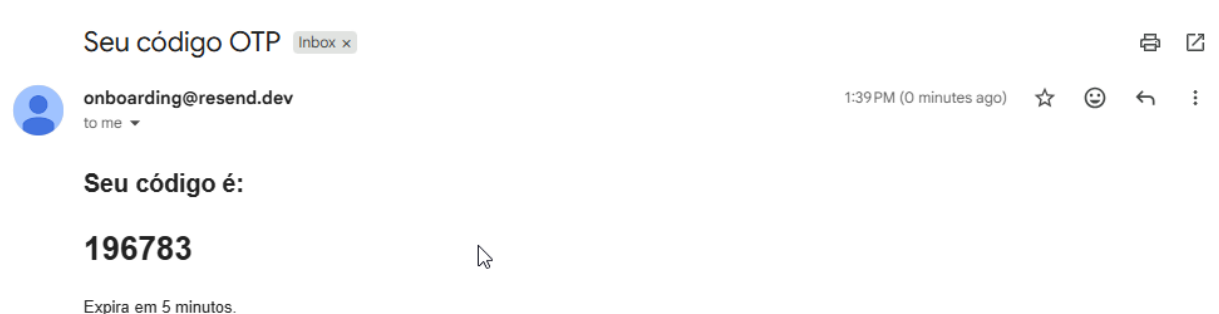


Figura 13 – Validação Token

Fonte: Elaborado pelo autor (2026)



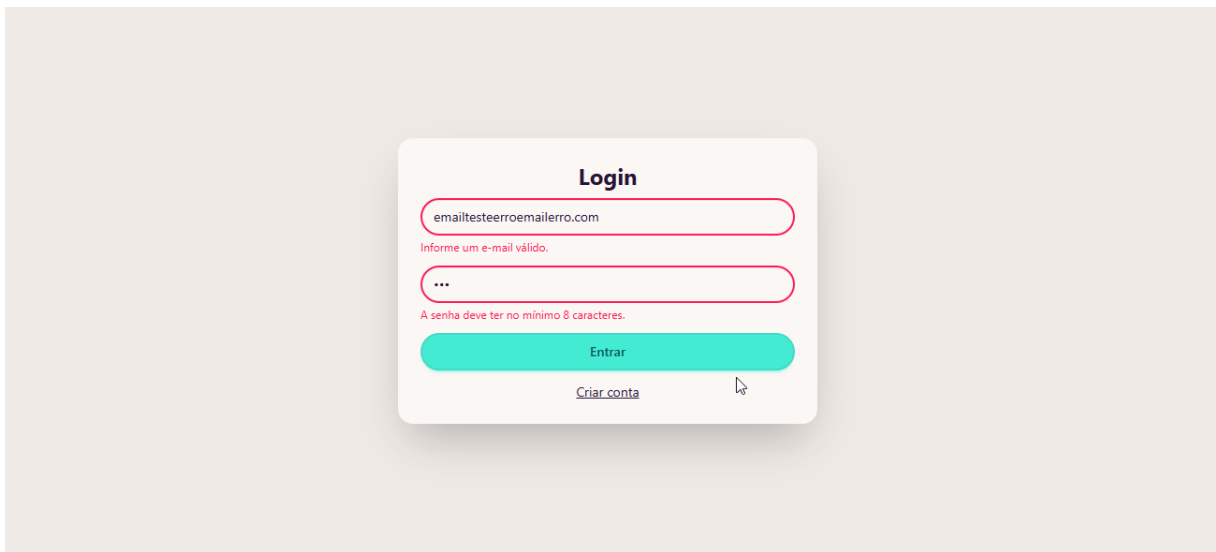
A interface de validação token é apresentada em uma caixa centralizada sobre um fundo cinza claro. No topo, o título "Verificação" está em negrito. Abaixo dele, o texto "Digite o código enviado para:" precede o e-mail "mths8844@gmail.com". Um campo de entrada arredondado contém o número "196783". Na base, um botão verde com o texto "Verificar" e um ícone de cursor de mouse indica a ação a ser realizada.

Validação de Formulários

Os formulários de cadastro e login foram desenvolvidos com validação de campos no frontend, exibindo mensagens de erro diretamente abaixo do campo correspondente, com destaque visual em vermelho. No cadastro, são validados o formato do e-mail, o tamanho mínimo da senha de oito caracteres e a correspondência entre senha e confirmação de senha. No login, são validados o formato do e-mail e o tamanho mínimo da senha. O envio da requisição ao servidor só ocorre após todos os campos passarem pela validação.

Figura 14 – validação de Erro Cadastro

Fonte: Elaborado pelo autor (2026)

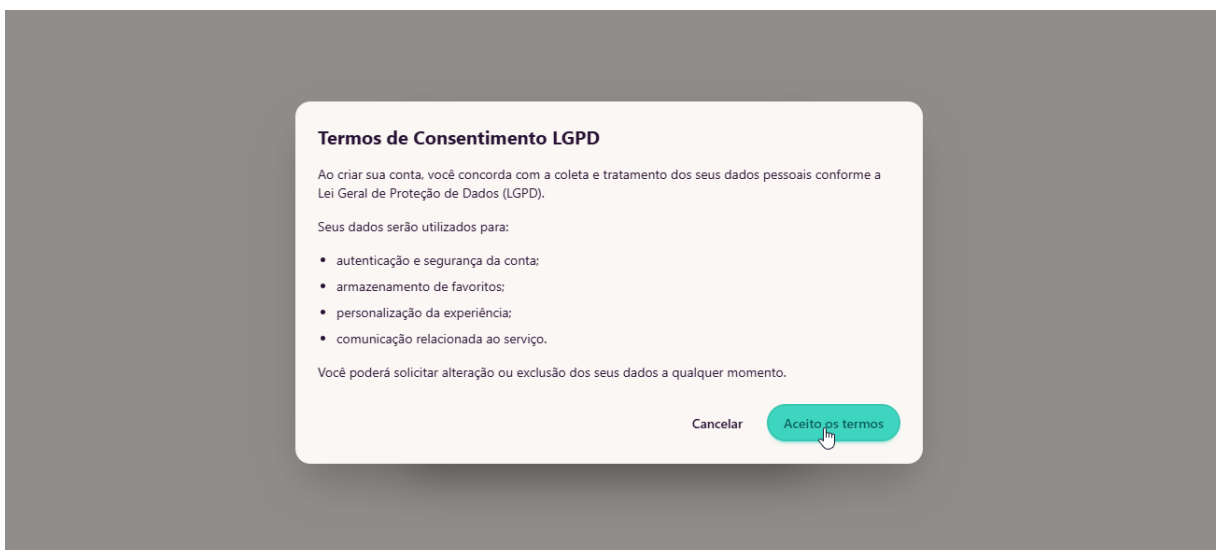


Consentimento LGPD

Durante o processo de cadastro, o sistema exibe obrigatoriamente um modal de consentimento de uso de dados conforme a Lei Geral de Proteção de Dados (LGPD). O usuário é informado sobre a finalidade do tratamento de seus dados pessoais e deve aceitar os termos antes de concluir o cadastro. Caso o usuário cancele, o cadastro não é realizado.

Figura 15 – Ciência lei LGPD, termos de uso

Fonte: Elaborado pelo autor (2026)



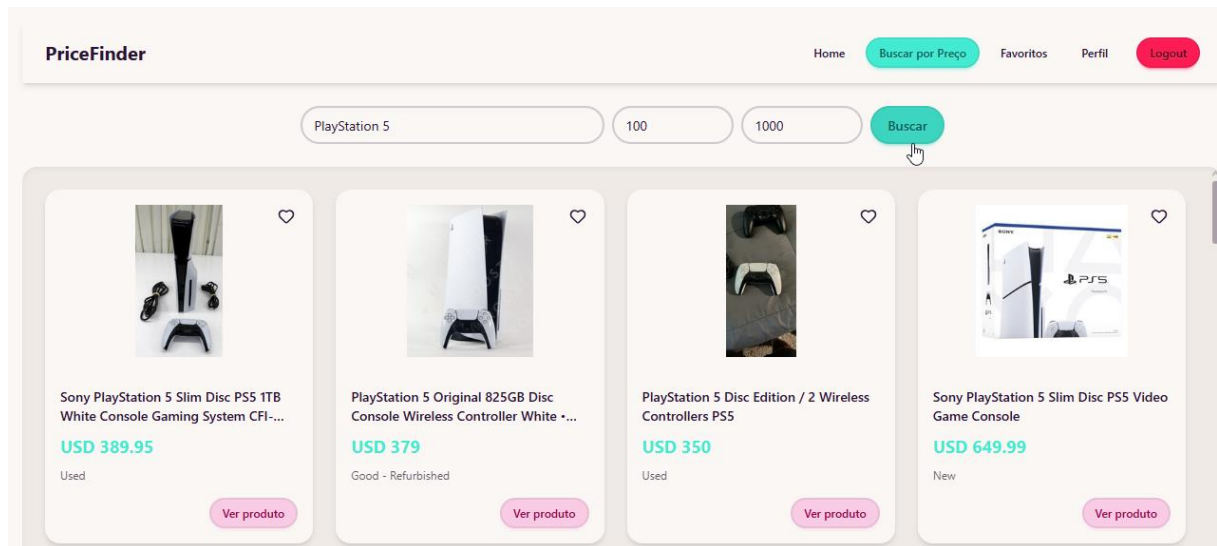
Busca e Comparação de Preços

A funcionalidade principal do sistema permite que o usuário autenticado realize buscas de produtos por palavra-chave, visualizando os resultados em formato

de cards contendo imagem, título, preço em dólares, condição do produto e botão de acesso ao produto. Os resultados são provenientes da API do eBay, integrada ao backend por meio do fluxo OAuth 2.0. Ao clicar em "Ver produto", o usuário é redirecionado para a página do produto na plataforma do eBay.

Figura 16 – Busca de Produto

Fonte: Elaborado pelo autor (2026)

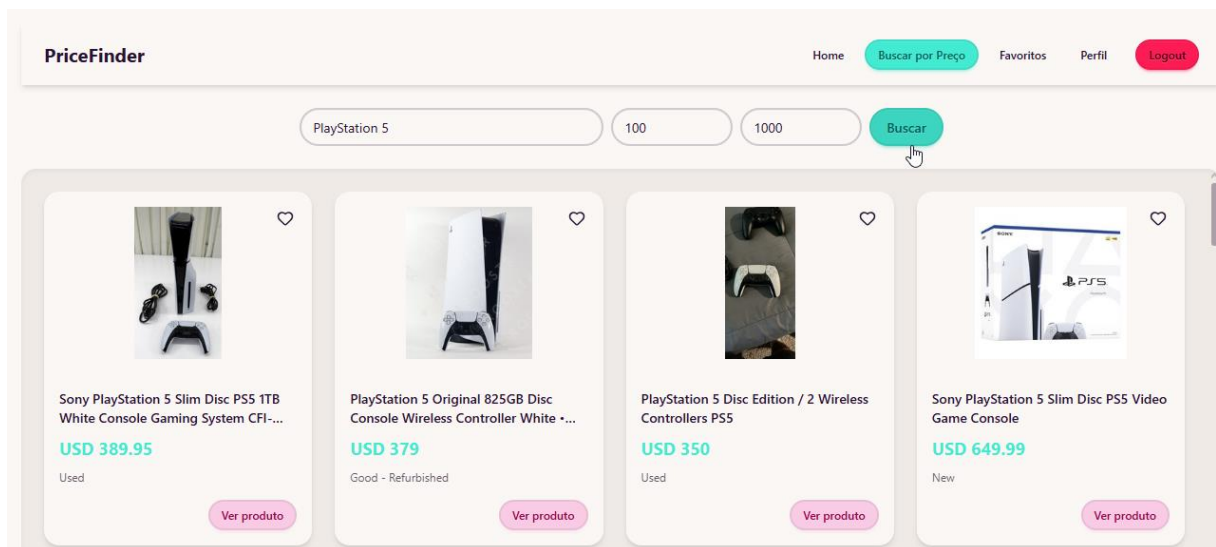


Busca por Faixa de Preço

A tela "Buscar por Preço" permite que o usuário combine a busca por palavra-chave com filtros de preço mínimo e máximo, refinando os resultados exibidos conforme a faixa de valores desejada

Figura 17 – Busca por preço

Fonte: Elaborado pelo autor (2026)



Modal de Detalhes do Produto

Ao clicar em um card de produto, o sistema exibe um modal contendo informações detalhadas, incluindo imagem ampliada, título completo, preço, condição, indicador de frete grátis e botão de redirecionamento direto para a página do produto no eBay.

Figura 18 – Detalhes do produto

Fonte: Elaborado pelo autor (2026)

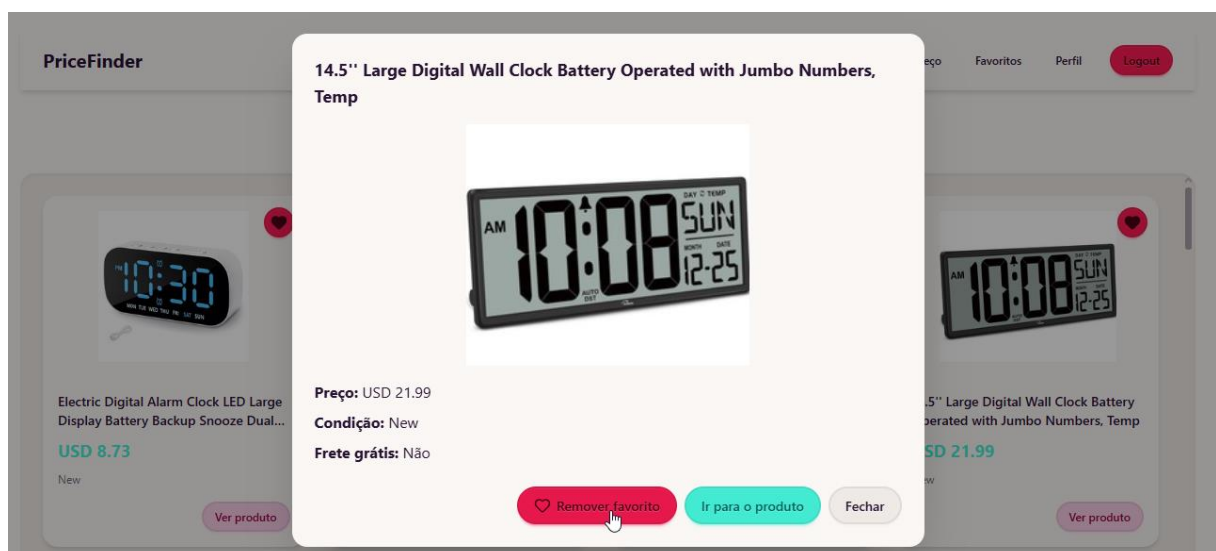
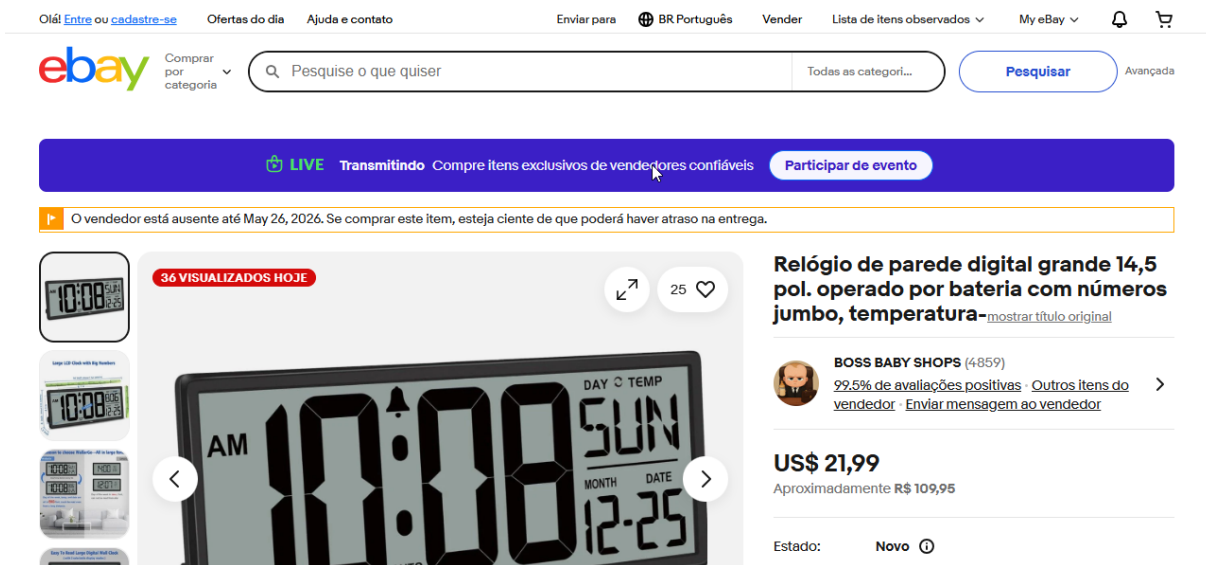


Figura 19 – Ebay

Fonte: Elaborado pelo autor (2026)



Sistema de Favoritos

O sistema permite que o usuário autenticado salve produtos como favoritos diretamente pelos cards ou pelo modal de detalhes. Os produtos salvos ficam disponíveis na tela "Meus Favoritos", que exibe o título, link do produto e data de salvamento. O usuário pode remover favoritos individualmente ou acessar o produto diretamente pelo botão "Abrir produto".

Figura 20 – Adicionar aos favoritos

Fonte: Elaborado pelo autor (2026)

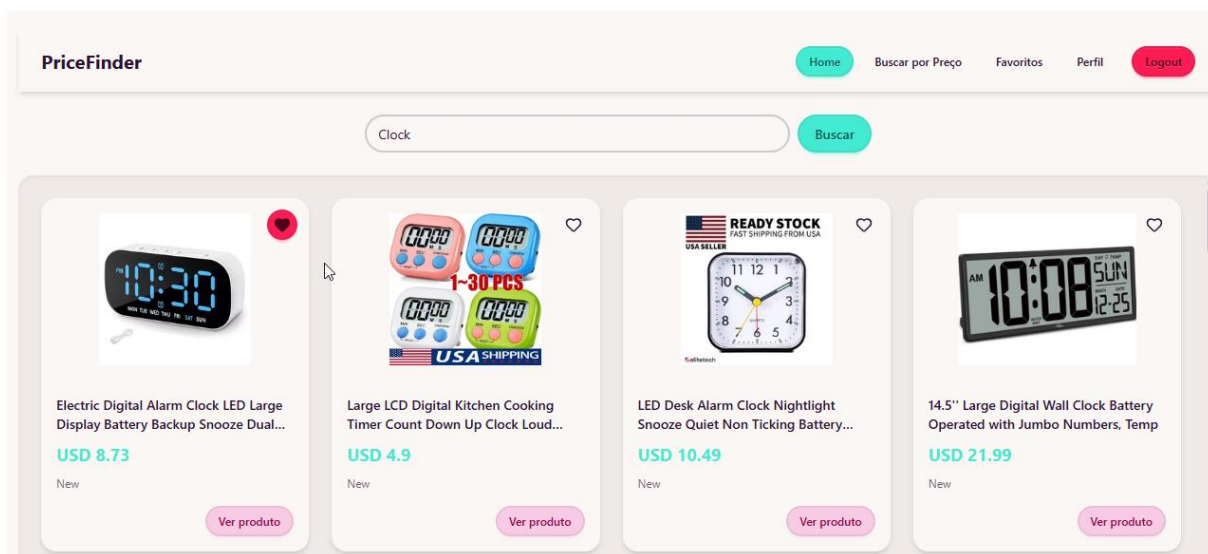
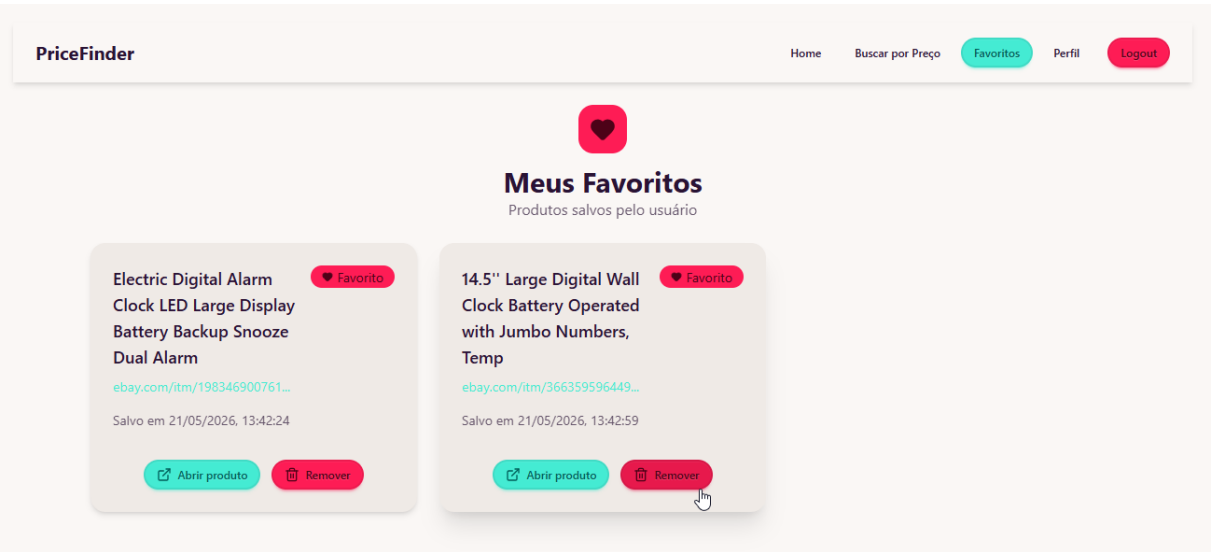


Figura 21 – Favoritos

Fonte: Elaborado pelo autor (2026)



Perfil do Usuário e Exclusão de Conta

A tela de perfil exibe as informações cadastradas do usuário, como primeiro nome, sobrenome e e-mail. Nessa tela também está disponível a funcionalidade de exclusão permanente de conta, localizada na seção "Zona de Perigo". Para confirmar a exclusão, o usuário deve digitar a palavra "EXCLUIR" no campo correspondente antes de acionar o botão, evitando exclusões acidentais.

Figura 22 – Perfil

Fonte: Elaborado pelo autor (2026)

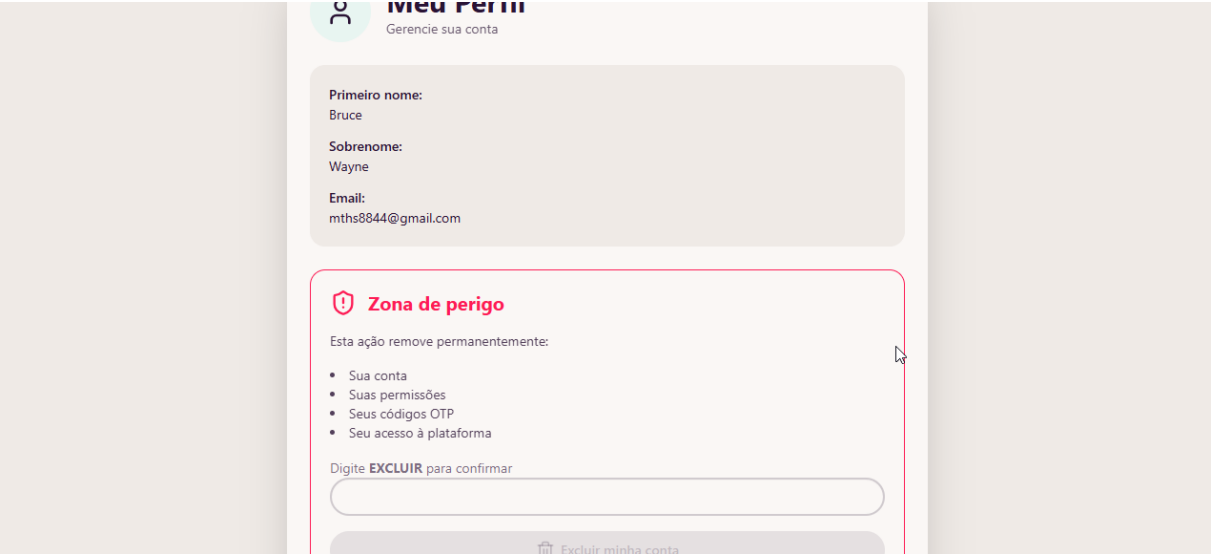


Figura 23 – Exclusão de conta

Fonte: Elaborado pelo autor (2026)



Primeiro nome:
Bruce

Sobrenome:
Wayne

Email:
mths8844@gmail.com

! Zona de perigo

Esta ação remove permanentemente:

- Sua conta
- Suas permissões
- Seus códigos OTP
- Seu acesso à plataforma

Digite **EXCLUIR** para confirmar

EXCLUIR

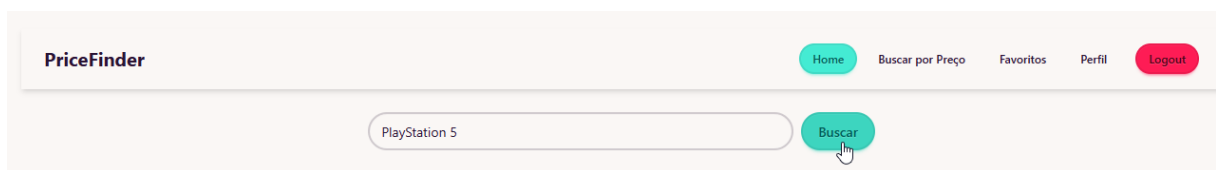
Excluir minha conta

Logout

O sistema disponibiliza o botão de logout na barra de navegação, encerrando a sessão do usuário e invalidando o token JWT armazenado localmente, redirecionando-o para a tela de login.

Figura 24 – Botão de logout

Fonte: Elaborado pelo autor (2026)



REFERÊNCIAS BIBLIOGRÁFICAS

BANKS, Alex; PORCELLO, Eve. Learning React: Modern Patterns for Developing React Apps. 2. ed. Sebastopol: O'Reilly Media, 2020.

DEITEL, Paul; DEITEL, Harvey. Java: Como Programar. 10. ed. São Paulo: Pearson, 2017.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. JSON Web Token (JWT). IETF RFC 7519, 2015.

STONEBRAKER, Michael; ROWE, Lawrence. The Design of POSTGRES. Proceedings of the 1986 ACM SIGMOD Conference.

WALLS, Craig. Spring Boot in Action. 2. ed. Manning Publications, 2022.

LAUDON, Kenneth C.; TRAVER, Carol Guercio. E-commerce 2018: Business, Technology, Society. 14. ed. Boston: Pearson, 2018.

SALVADOR, Maurício. Gerente de E-commerce. São Paulo: Ecommerce School, 2013.